

[EPK14,CCS] RAPPOR: Randomized Aggregatable Privacy-Preserving Ordinal Response

SUMMARY: Based on random response, this paper proposes a user data collection protocol that satisfies local DP.

1 Background

User data collection is important for internet tech. companies. With these data, they can iteratively optimize users' experience. However, collecting users' data may undermine their privacy, thus violates privacy laws such as European GDPR.

This work is deployed in Chromium, which is the most popular internet explorer. The developers claim that DP is used on a large scale for the first time in practical application.

2 Setting & Goals

This paper's challenge is threefold:

[C1] First, the server belongs to companies is untrusted; therefore, the protocol must be local DP.

[C2] Second, the data collected/submitted by users may be arbitrary length.

[C3] Third and probably the most important, the data of a client may be repeatedly (or even infinitely) uploaded to the server. This will consume the privacy budget infinitely.

3 Methodology | Roadmap

To preserve user's privacy, a naive solution is to use random response before a client uploads data to the server. However, this solution does not solve [C2] and [C3].

To resolve [C2], RAPPOR uses bloom filter to process data which can handle inputs in arbitrary length.

To resolve [C3], RAPPOR uses random response twice, one for repeatedly query and other for one-time query.

The workflow of RAPPOR is as follows:

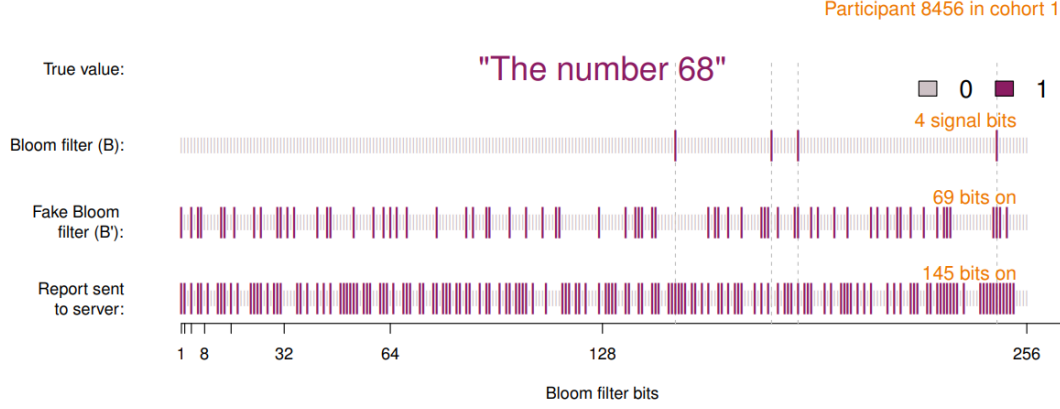


Figure 1: Life of a RAPPOR report: The client value of the string “The number 68” is hashed onto the Bloom filter B using h (here 4) hash functions. For this string, a Permanent randomized response B' is produced and memoized by the client, and this B' is used (and reused in the future) to generate Instantaneous randomized responses S (the bottom row), which are sent to the collecting service.

1. **Signal.** Hash client's value v onto the Bloom filter B of size k using h hash functions.
2. **Permanent randomized response.** For each client's value v and bit $i, 0 \leq i < k$ in B , create a binary reporting value B'_i which equals to

$$B'_i = \begin{cases} 1, & \text{with probability } \frac{1}{2}f \\ 0, & \text{with probability } \frac{1}{2}f \\ B_i, & \text{with probability } 1 - f \end{cases}$$

where f is a user-tunable parameter controlling the level of longitudinal privacy guarantee.

Subsequently, this B' is memoized and reused as the basis for all future reports on this distinct value v .

3. **Instantaneous randomized response.** Allocate a bit array S of size k and initialize to 0. Set each bit i in S with probabilities

$$P(S_i = 1) = \begin{cases} q, & \text{if } B'_i = 1. \\ p, & \text{if } B'_i = 0. \end{cases}$$

4. **Report.** Send the generated report S to the server.

Client Encode

- Estimate the number of times each bit i within cohort j , t_{ij} , is truly set in B for each cohort. Given the number of times each bit i in cohort j , c_{ij} was set in a set of N_j reports, the estimate is given by

$$t_{ij} = \frac{c_{ij} - (p + \frac{1}{2}fq - \frac{1}{2}fp)N_j}{(1 - f)(q - p)}.$$

Let Y be a vector of t_{ij} 's, $i \in [1, k], j \in [1, m]$.

- Create a design matrix X of size $km \times M$ where M is the number of candidate strings under consideration. X is mostly 0 (sparse) with 1's at the Bloom filter bits for each string for each cohort. So each column of X contains hm 1's at positions where a particular candidate string was mapped to by the Bloom filters in all m cohorts. Use Lasso [26] regression to fit a model $Y \sim X$ and select candidate strings corresponding to non-zero coefficients.
- Fit a regular least-squares regression using the selected variables to estimate counts, their standard errors and p-values.
- Compare p-values to a Bonferroni corrected level of $\alpha/M = 0.05/M$ to determine which frequencies are statistically significant from 0. Alternatively, controlling the False Discovery Rate (FDR) at level α using the Benjamini-Hochberg procedure [3], for example, could be used.

Server Decode

Hash collisions. RAPPOR uses Bloom filter to collect data from users, thus the collected data may corresponds to multiple answers. (Since one bit in BF have multiple candidate pre-image items.) Therefore, RAPPOR provides different BF to different users.

Parameter selection. Through the selection of different parameters, RAPPOR can achieve any privacy level between full privacy and zero privacy. When consider the utility of RAPPOR, the best choice of the number of hash function is two. Meanwhile, the number of cohorts and the size of BF do not show a clear correlation to the utility.

4 Evaluation

In the below figure, the authors presents the accuracy of RAPPOR under different maginitude of users, where the green and black and represents noised and real view respectively.

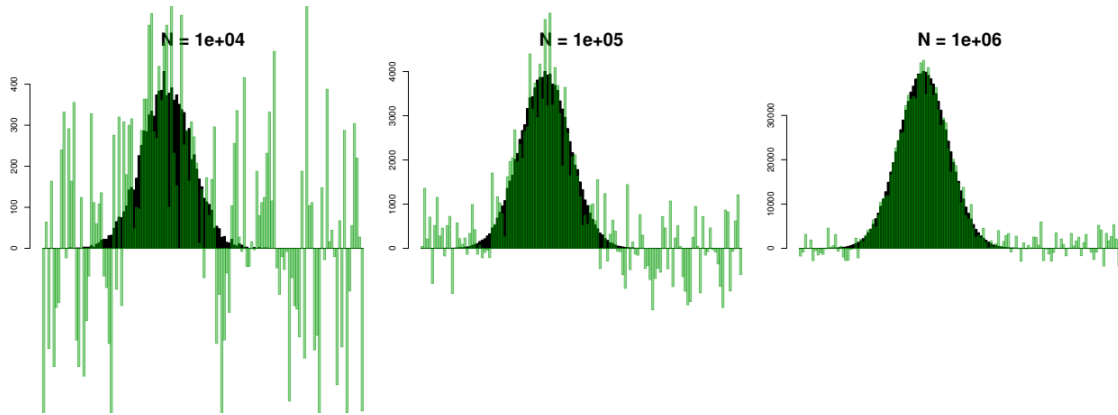


Figure 4: Simulations of learning the normal distribution with mean 50 and standard deviation 10. The RAPPOR privacy parameters are $q = 0.75$ and $p = 0.5$, corresponding to $\epsilon = \ln(3)$. True sample distribution is shown in black; light green shows the estimated distribution based on the decoded RAPPOR reports. We do not assume *a priori* knowledge of the Normal distribution in learning. If such prior information were available, we could significantly improve upon learning the shape of the distribution via smoothing.

5 Observation & Insights

The framework of RAPPOR is informative for our design of impact tracking.

- First, the permanent randomized response may help us to resolve the problem of multiple reports.
- Second, the functionality of our membership function is the same as the decode algorithm in RAPPOR, which derives true positives from a noised view.
- Third, this paper devotes only 1 page to describe the algorithm of RAPPOR; the rest of pages is devoted to anlysis, experiments and discussion.